Ejercicio obligatorio

Sistemas Operativos 2026

Universidad de Montevideo – Facultad de Ingeniería

Parte 1: comando `ent`

Implementar en lenguaje C un programa de línea de comandos llamado `ent`, capaz de operar con enteros de longitud arbitraria.

En esta primera versión, el programa deberá soportar únicamente la operación `sumar`, aplicada a uno o más enteros no negativos expresados en base decimal.

Los operandos no podrán almacenarse en tipos numéricos básicos de C (`int`, `long`, `long long`, etc.), sino que deberán representarse mediante una estructura definida por el estudiante, utilizando memoria dinámica.

Esta primera parte deberá resolverse en **un único archivo fuente**.

NOTA: en breve se entregará la consigna de la Parte 2, que agrega más operaciones: resta, multiplicación, además de que separaremos el proyecto en múltiples archivos fuente y usaremos un Makefile para simplificar la compilación y generación del ejecutable.

Sintaxis

```
ent sumar N1 N2 [N3 ...]
```

donde `N1`, `N2`, `N3`, etc. son enteros no negativos escritos en decimal.

Ejemplo

```
ent sumar 32498923321 321938192123 23284932849328
```

debe producir:

```
23639369964772
```

Requisitos

1. El programa debe validar la cantidad de argumentos recibidos.
2. El programa debe validar que la operación solicitada sea `sumar`.
3. Cada operando debe validarse como entero no negativo en base decimal.
4. Debe definirse una estructura para representar enteros de longitud arbitraria.
5. La memoria para representar los números debe ser dinámica.
6. El programa puede asumir un tamaño máximo fijo para la línea de entrada.
7. La suma debe implementarse manualmente, sin convertir los operandos a tipos numéricos nativos.
8. El resultado debe imprimirse en decimal.
9. Si ocurre un error de entrada o procesamiento, el programa debe informarlo.

Estructura sugerida

```
typedef struct {
   char *digits;   // decimal digits 0..9, stored in reverse order
   int len;        // number of digits
   int sign;       // 1 for positive, -1 for negative
} Bigint;
```

Sugerencia de funciones

Se sugiere descomponer el problema en funciones. Por ejemplo:

```
Bigint *bigint_new(const char *text);
void bigint_print(const Bigint *n);
void bigint_free(Bigint *n);
Bigint *bigint_add(const Bigint *a, const Bigint *b);
```

Sugerencia de funciones auxiliares

Para organizar la solución, puede ser útil dividir el problema en funciones. A modo orientativo, se sugieren las siguientes:

```
int is_decimal_string(const char *text);
```

Verifica si una cadena representa un entero no negativo válido en base decimal.

Argumentos

- `text`: cadena a validar

Valor de retorno

- `1` si la cadena contiene solo caracteres entre `'0'` y `'9'` y no es vacía
- `0` en caso contrario

```
Bigint *bigint_new(const char *text);
```

Crea un `Bigint` a partir de una cadena de texto válida.

Argumentos

- `text`: cadena que contiene el número en decimal

Valor de retorno

- puntero a un `Bigint` válido si la conversión es correcta
- `NULL` si hay error de formato o memoria

```
void bigint_free(Bigint *n);
```

Libera la memoria asociada a un `Bigint`.

Argumentos

- `n`: puntero a un `Bigint`

Valor de retorno

- ninguno

```
void bigint_print(const Bigint *n);
```

Imprime el número en formato decimal en la salida estándar.

Argumentos

- `n`: puntero a un `Bigint`

Valor de retorno

- ninguno

```
int bigint_max_len(const Bigint *a, const Bigint *b);
```

Devuelve la mayor longitud entre dos `Bigint`.

Argumentos

- `a`: primer operando
- `b`: segundo operando

Valor de retorno

- la mayor de las dos longitudes

```
char bigint_digit_at(const Bigint *n, int pos);
```

Obtiene el dígito en una posición dada. Si la posición excede la longitud del número, devuelve `0`.

Argumentos

- `n`: puntero a un `Bigint`
- `pos`: posición (empezando en 0)

Valor de retorno

- valor del dígito (0–9), o `0` si está fuera de rango

```
Bigint *bigint_add(const Bigint *a, const Bigint *b);
```

Calcula la suma de dos `Bigint`.

Argumentos

- `a`: primer operando
- `b`: segundo operando

Valor de retorno

- puntero al resultado
- `NULL` en caso de error

Observación

No es necesario implementar todas estas funciones. Esta lista es solo una guía para ayudar a estructurar la solución en partes más simples.

Consideraciones

- En esta etapa, un entero válido contiene únicamente caracteres entre `'0'` y `'9'`.
- Se podrán aceptar operandos con ceros a la izquierda, pero la representación interna deberá quedar normalizada.
- El cero deberá tener una representación única.
- Se sugiere implementar primero una versión que funcione correctamente con dos operandos, y luego extenderla a múltiples operandos.
- No se permite usar bibliotecas de enteros grandes.

Entrada y salida

- El resultado debe imprimirse exclusivamente como un número decimal, sin texto adicional.
- El resultado debe imprimirse en la salida estándar (`stdout`) seguido de un salto de línea.
- Los mensajes de error deben enviarse a la salida de error (`stderr`).

Casos de prueba recomendados

Se recomienda probar el programa con casos que cubran tanto situaciones válidas como inválidas.

1. Sumas simples

ent sumar 2 3

Salida esperada:

5

---

ent sumar 10 20

Salida esperada:

30

---

2. Suma con acarreo

ent sumar 9 1

Salida esperada:

10

---

ent sumar 99 1

Salida esperada:

100

---

ent sumar 9999 1

Salida esperada:

10000

---

3. Ceros

ent sumar 0 0

Salida esperada:

0

---

ent sumar 0 12345

Salida esperada:

12345

---

ent sumar 0000 000

Salida esperada:

0

---

ent sumar 00012 003

Salida esperada:

15

---

4. Más de dos operandos

ent sumar 1 2 3

Salida esperada:

6

---

ent sumar 10 20 30 40

Salida esperada:

100

---

ent sumar 999 1 1

Salida esperada:

1001

---

5. Números grandes

ent sumar 32498923321 321938192123

Salida esperada:

354437115444

---

ent sumar 32498923321 321938192123 23284932849328

Salida esperada:

23639369964772

---

ent sumar 12345678901234567890 98765432109876543210

Salida esperada:

111111111011111111100

---

6. Operación inválida

ent restar 2 3

Comportamiento esperado:

- debe informar error
- no debe imprimir un número en `stdout`

---

7. Cantidad incorrecta de argumentos

ent

Comportamiento esperado:

- debe informar error

---

ent sumar

Comportamiento esperado:

- debe informar error

---

ent sumar 123

Comportamiento esperado:

- si el programa exige al menos dos operandos, debe informar error

---

8. Operandos inválidos

ent sumar a 3

Comportamiento esperado:

- debe informar error

---

ent sumar 12x 3

Comportamiento esperado:

- debe informar error

---

ent sumar -5 2

Comportamiento esperado:

- debe informar error

---

ent sumar +7 2

Comportamiento esperado:

- debe informar error en esta etapa, ya que solo se aceptan dígitos entre `'0'` y `'9'`

---

ent sumar 3.14 2

Comportamiento esperado:

- debe informar error

---

9. Casos cercanos al límite de tamaño

Aquí conviene probar operandos muy largos, cercanos al máximo admitido por la implementación.

Por ejemplo, si `MAX_DIGITS` vale 1000:

- un operando de 1000 dígitos válidos
- una suma cuyo resultado tenga 1000 dígitos
- una suma cuyo resultado requiera 1001 dígitos

Comportamiento esperado:

- si el operando cabe, debe procesarse correctamente
- si el operando o el resultado exceden la capacidad máxima, debe informarse error

---

10. Composición en shell

Dado que la salida estándar debe contener únicamente el número, conviene probar casos como:

echo $(ent sumar 100 200)

Salida esperada:

300

---

ent sumar $(ent sumar 1 2) 4

Salida esperada:

7

Observación

En los casos de error, el texto exacto del mensaje puede variar según la implementación, salvo que se indique lo contrario en el enunciado. Lo importante es que:

- el error se informe por `stderr`
- no se imprima un resultado numérico incorrecto en `stdout`

📊 Rúbrica de corrección – Parte 1 (`ent sumar`)

Total: **10 puntos**

---

1. Uso correcto del programa (1 punto)

- (1.0) La sintaxis es correcta: `ent sumar N1 N2 [...]`
- (0.5) Funciona parcialmente (por ejemplo, solo con dos operandos)
- (0.0) No respeta la sintaxis

---

2. Validación de argumentos (1 punto)

- (1.0) Detecta correctamente:
  - operación inválida
  - cantidad insuficiente de operandos
- (0.5) Detecta algunos errores pero no todos
- (0.0) No valida

---

3. Validación de operandos (1 punto)

- (1.0) Rechaza correctamente:
  - caracteres no numéricos
  - signos (`+`, `-`)
  - strings inválidos
- (0.5) Valida parcialmente
- (0.0) No valida

---

4. Representación del Bigint (1 punto)

- (1.0) Implementación correcta:
  - dígitos en orden invertido
  - manejo correcto de `len`
- (0.5) Funciona pero con detalles discutibles
- (0.0) Uso incorrecto o uso de tipos nativos

---

5. Uso de memoria dinámica (1.5 puntos)

- (1.5) Usa `malloc` correctamente y libera memoria
- (1.0) Usa memoria dinámica pero con problemas (fugas, etc.)
- (0.0) No usa memoria dinámica

---

6. Implementación de la suma (1.5 puntos)

- (1.5) Correcta en todos los casos:
  - acarreo
  - operandos de distinta longitud
- (1.0) Funciona en casos simples pero falla en edge cases
- (0.0) Incorrecta

---

7. Manejo de múltiples operandos (1 punto)

- (1.0) Suma correctamente más de dos operandos
- (0.5) Solo funciona con dos (pero bien implementado)
- (0.0) No funciona

---

8. Normalización y casos especiales (0.5 puntos)

- (0.5) Maneja correctamente:
  - ceros a la izquierda
  - resultado `0`
- (0.25) Manejo parcial
- (0.0) Incorrecto

---

9. Salida estándar correcta (1 punto)

- (1.0) Imprime **solo el número** en `stdout`
- (0.5) Tiene texto extra pero el resultado es correcto
- (0.0) Formato incorrecto

---

10. Manejo de errores (stderr) (0.5 puntos)

- (0.5) Errores van a `stderr` y no contaminan `stdout`
- (0.25) Manejo parcial
- (0.0) Mezcla salida o no informa errores

🧠 Preguntas para la defensa

1. Estructura general de un programa en C

- Describa la estructura general de un programa en C.
- Explique el rol de las directivas del preprocesador, en particular `#include`.
- Explique las etapas involucradas en la construcción y ejecución de un programa en C:
    - preprocesamiento
    - compilación
    - enlace (linking)
    - Ejecución
    - Describa brevemente qué ocurre en cada una de estas etapas.
    - ¿Qué tipo de errores pueden detectarse en cada etapa?

---

2. Función `main` y argumentos

- ¿Qué función cumple `main` en un programa en C?
- Explique el significado de `argc` y `argv`.
- ¿Cómo se representan los argumentos de línea de comandos en memoria?
- ¿Qué representa el valor de retorno de `main` y cómo se utiliza en el sistema operativo?

---

3. Tipos de datos y variables

- Describa los tipos básicos del lenguaje C.
- Compare variables locales y variables globales:
    - ¿Dónde se almacenan?
    - ¿Cuál es su tiempo de vida?
    - ¿En qué casos usaría cada una?
- ¿Qué problemas pueden surgir al abusar de variables globales?

---

4. Memoria: stack vs heap

- Explique la diferencia entre memoria automática (stack) y memoria dinámica (heap).
- ¿Qué tipo de variables se almacenan en cada una?
- ¿Qué hace la función `malloc`?
- ¿Qué ocurre si no se libera la memoria reservada con `malloc`?
- ¿Qué errores pueden ocurrir al manejar memoria dinámica?

---

5. Punteros

- Explique qué es un puntero.
- ¿Qué contiene una variable puntero?
- Explique el significado de los operadores `*` y `&`.
- ¿Qué diferencia hay entre declarar y desreferenciar un puntero?
- ¿Qué problemas pueden surgir al usar punteros incorrectamente?

---

6. Relación entre punteros y arrays

- Explique la relación entre arrays y punteros en C.
- ¿Qué significa que un array “decay” a puntero?
- Compare:

  ```
  char digits[100];
  char *digits;
  ```

- ¿Por qué en el ejercicio obligatorio se utiliza memoria dinámica para los dígitos?

---

7. Estructuras (`struct`)

- ¿Qué es una estructura en C y para qué se utiliza?
- ¿Cómo se accede a sus campos?
- Explique la diferencia entre:

  ```
  n.len
  n->len
  ```

- ¿Qué representa la estructura `Bigint` en el ejercicio obligatorio?

---

8. Representación de enteros grandes

- Explique cómo se representa un número en la estructura `Bigint`.
- ¿Por qué los dígitos se almacenan en orden invertido?
- ¿Cómo se representaría internamente el número 123?
- ¿Cómo se representaría el número 1000?
- ¿Qué ventajas tiene esta representación para implementar operaciones?

---

9. Algoritmo de suma

- Describa el algoritmo utilizado para sumar dos números grandes.
- ¿Qué es el acarreo (carry) y cómo se maneja?
- ¿Cómo se suman números de distinta longitud?
- ¿Por qué se reserva espacio adicional en el resultado?
- ¿Qué ocurre si hay un acarreo en el último dígito?

---

10. Manejo de memoria en el ejercicio

- ¿Dónde se reserva memoria para la estructura `Bigint`?
- ¿Dónde se reserva memoria para el array de dígitos?
- ¿Quién es responsable de liberar la memoria?
- ¿Qué consecuencias tiene no liberar la memoria correctamente?

---

11. Validación de entrada

- ¿Cómo valida su programa que un operando es correcto?
- ¿Qué casos de entrada inválida contempla?
- ¿Por qué en esta etapa no se permiten signos?
- ¿Qué ocurriría si el usuario ingresa caracteres no numéricos?

---

12. Entrada/salida y uso en shell

- Explique la diferencia entre `stdout` y `stderr`.
- ¿Por qué el programa debe imprimir únicamente el número en la salida estándar?
- ¿Qué ocurre al ejecutar:

  $(ent sumar 100 200)

- ¿Qué problemas aparecerían si el programa imprimiera texto adicional?

---

13. Comprensión del propio código

- Seleccione una función de su implementación y explíquela línea por línea.
- ¿Qué decisiones de diseño tomó y por qué?
- ¿Qué parte del programa le resultó más compleja?
- Si tuviera que mejorar su solución, ¿qué cambiaría?
- Señale un posible error (bug) que podría aparecer y cómo lo detectaría.

🎯 Nota final para el estudiante

El objetivo de esta instancia es verificar que el código entregado ha sido comprendido.

Un programa que funciona correctamente, pero que no puede ser explicado por quien lo entrega, no será considerado una solución válida.